

SALES DATA WAREHOUSE

SQL DATABASE DOCUMENTATION

<https://drive.google.com/drive/folders/11rllqp9jAcTz7KhgF-JmF36GQITqkFOj?usp=sharing>

Fact & Dimension Tables • Views • Stored Procedures • Analytical Reports • 15 Advanced Queries

DOCUMENT PURPOSE	Explains the supplied MySQL sales warehouse code, its structure, analytical logic, and business purpose.
MODEL	Star schema with one central fact table and four descriptive dimensions.

1. System Overview

The script implements a sales data warehouse using a star-schema design. **fact_sales** stores transaction measures, while customer, product, location, and date dimensions provide descriptive context. Reusable views, a stored procedure, CTE-based reports, and advanced queries support business intelligence and decision-making.

TABLE	TYPE	PURPOSE	PRIMARY KEY
dim_customer	Dimension	Customer identity and segment	customer_id
dim_product	Dimension	Product hierarchy and category	product_id
dim_location	Dimension	Country, state, city, region	location_id
dim_date	Dimension	Calendar and time attributes	date_id
fact_sales	Fact	Orders, measures, and dimension links	fact_id

2. Fact & Dimension Tables

2.1 dim_customer

Customer master data. **customer_id** is the primary key. The table stores name and business segment.

2.2 dim_product

Product master data. **product_id** is the primary key, with product name, category, and sub-category.

2.3 dim_location

Geographic dimension. **location_id** is an AUTO_INCREMENT key; country, state, city, postal code, and region describe the location.

2.4 dim_date

Calendar dimension. **date_id** is the warehouse key. full_date plus month, month_name, quarter, and year support time analysis.

2.5 fact_sales

Central transaction table containing order/customer/product/location/date references plus ship_mode, sales, quantity, discount, and profit.

3. Relationships

FOREIGN KEY	REFERENCES
-------------	------------

fact_sales.customer_id	dim_customer.customer_id
fact_sales.product_id	dim_product.product_id
fact_sales.location_id	dim_location.location_id
fact_sales.order_date_id	dim_date.date_id
fact_sales.ship_date_id	dim_date.date_id

Using dim_date twice allows the same dimension to analyze both the **order date** and the **ship date**.

4. Views & Stored Procedure

OBJECT	WHAT IT DOES	KEY OUTPUT
vw_monthly_kpis	Aggregates fact_sales by year/month.	Orders, revenue, profit, average discount %
sp_kpi_by_date_range	Filters a supplied date range and groups by product category.	Orders, sales, profit, profit margin %

KPI calculations: total_orders = COUNT(DISTINCT order_id); revenue = SUM(sales); profit = SUM(profit); average discount = AVG(discount) × 100.

Example procedure call

```
CALL sp_kpi_by_date_range('2025-01-01', '2025-12-31');
```

5. Analytical Reports

1. Profitability Analysis Report	CTE RegionalProfitability aggregates sales and profit by region, category, and sub-category. Status = Unprofitable Line if profit < 0; Low Margin if profit/sales < 15%; otherwise High Margin. Business use: Find weak product lines and regional profitability.
2. Customer Behavior & Loyalty Tier Report	CTE CustomerRFM calculates distinct orders, lifetime sales, and lifetime profit. VIP = 5+ orders and 3000+ sales; Repeat = 2–4 orders; otherwise One-Time. Business use: Support customer segmentation and loyalty decisions.
3. Sales Trends & MoM Growth Report	CTE MonthlyKPIs creates monthly KPIs. LAG retrieves the previous month's revenue and the query calculates month-over-month growth. Business use: Monitor growth, decline, revenue, orders, and profit over time.

6. Advanced Query Suite — 15 Queries

#	QUERY	LOGIC	BUSINESS PURPOSE
01	High Discount Impact	Filters discount > 20%, then summarizes sales and profit by category.	Evaluate the financial effect of heavy discounting.
02	Unprofitable Sub-Categories	Groups profit by sub-category and keeps negative-profit groups.	Find loss-making sub-categories.
03	Regional Margin Breakdown	Aggregates sales/profit by region and labels Gain or Loss.	Compare regional financial performance.
04	Average Shipping Speed	DATEDIFF(ship date, order date), averaged by shipping mode.	Compare delivery speed.
05	Top 10 Spending Customers	Ranks customers by SUM(sales) and returns 10.	Identify highest-value customers.
06	Revenue Share by Segment	Sums sales for every customer segment.	Measure segment contribution.
07	Products Above Average	Compares product-level SUM(sales) with overall AVG(sales).	Identify products exceeding the transaction-level average.
08	Consumer Top Spender	Filters Consumer customers and returns the highest sales total.	Find the biggest Consumer customer.
09	Yearly Trend	Aggregates sales and profit by year.	Review annual performance.
10	Popular Category by Quantity	Sums quantity by category and sorts descending.	Find the category with the most units sold.
11	Products in Multiple Regions	Counts distinct regions per product; keeps counts > 1.	Find geographically distributed products.
12	Cross-Selling Market Basket	Self-joins rows from the same order and pairs product IDs.	Find frequently co-purchased product pairs.
13	Bottom 5 States by Profit	Groups profit by state and returns the five lowest totals.	Locate weak-performing states.
14	Quarterly Financial Summary	Groups sales and profit by year and quarter.	Review quarterly financial results.
15	Loss Mitigation Flags	Filters profit < 0 and labels losses below -100 as Critical.	Prioritize loss investigation.

7. Technical Notes & Validation

MySQL syntax	AUTO_INCREMENT, DELIMITER, and LIMIT indicate MySQL-oriented syntax.
View ordering	ORDER BY inside a view should not be relied on for final presentation; apply ORDER BY when selecting from the view.
RFM terminology	CustomerRFM calculates frequency and monetary measures but not recency, so it is not a complete RFM model.
Query 7	SUM(sales) by product is compared with transaction-level AVG(sales); these are different aggregation levels.
Query 15	Because fact_sales is transaction-line based, an order can appear more than once if several lines have losses.
Query 12	product_id < product_id prevents reversed duplicates such as A-B and B-A.
Data quality	Validate duplicate IDs, missing dates, invalid discounts, and inconsistent customer/product keys during ETL.

8. Recommended Execution Order

1	Create the four dimension tables.
2	Create fact_sales after the dimensions so foreign keys can be created.
3	Load dimension data, especially dim_date for all order and shipping dates.
4	Load transactions into fact_sales.
5	Create KPI and analytical report views.
6	Create the stored procedure using the MySQL delimiter.
7	Run the 15 analytical queries and validate the results.

9. Overall Purpose

The complete SQL solution converts transactional sales data into an analytical warehouse. The dimensional model makes reporting easier, while the views and procedure provide reusable KPIs. The reports and advanced queries support decisions about profitability, customers, products, shipping, regions, sales trends, cross-selling, and loss mitigation.